

Viva: Project 20, Advice Generator

API: Advice Slip (<https://api.adviceslip.com>) - Key: No key needed

Brief: Tap a card to get a random piece of advice.

30 questions, roughly easy to hard. Sections A and B are common ground for every project; C and D are specific to this one. Ask 8 to 12 in a typical 10 minute viva.

A. Kotlin and Android basics

1. What is the difference between val and var, and which did you use for your Retrofit instance?
2. What is a data class, and what does Kotlin generate for you?
3. How did you model the advice response, given it nests a slip object?
4. What does ? mean on a property, and are any fields here genuinely optional?
5. Which Android component holds the current advice, and what happens on rotation?
6. How did you make the whole card tappable rather than just a button?
7. Where do you declare the INTERNET permission, and what happens if you forget it?

B. Retrofit, coroutines and networking

8. What problem does Retrofit solve for you?
9. Explain the role of the converter factory in your Retrofit builder.
10. This endpoint takes no parameters. What does your interface method look like?
11. Why is your API call function marked suspend?
12. Which dispatcher runs the network call, and which updates the UI?
13. What happens if you call the API on the main thread?
14. What is a coroutine scope, and which one did you launch from?
15. How do you handle the request failing?

C. The Advice Slip API and this app

16. What is the random advice endpoint, and what does the response look like?
17. The response wraps everything in a slip object with id and advice. How did you model that?
18. This API caches aggressively, so rapid taps can return the same advice. Why does that happen?
19. How could you defeat that caching, and is adding a random query parameter a good idea?
20. The API sometimes returns its JSON with a text/html content type. What does that do to your converter, and how did you fix it?
21. Is the API served over HTTPS, and why does that matter on Android?
22. What happens if the response body is empty or malformed?
23. How would you fetch advice by a specific id, and which endpoint does that?
24. There is also a search endpoint. What shape does it return, and how does it differ from random?

D. Edge cases, design and extensions

25. What does your app show while advice is loading?

26. A user has no connection and taps the card. Describe exactly what they see.
27. What stops the user firing many overlapping requests by tapping quickly?
28. How would you animate between the old and new advice rather than swapping instantly?
29. How would you save favourite advice so it survives app restarts?
30. How would you add a share button, and which Intent would you use?

Humble Coders project viva preparation. These are the kinds of questions you should be able to answer about the app you built. Knowing the list is not the same as knowing your own code.